Data Wrangling

Problem Statement: Data Wrangling on Real Estate Market Dataset: "RealEstate_Prices.csv" Description: The dataset contains information about housing prices in a specific real estate market. It includes various attributes such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to gain insights into the factors influencing housing prices and prepare the dataset for further analysis or modeling. Tasks to Perform:

1. Import the "RealEstate_Prices.csv" dataset. Clean column names by removing spaces, special characters, or renaming them for clarity.
2. Handle missing values in the dataset, deciding on an appropriate strategy (e.g., imputation or removal).
3. Perform data merging if additional datasets with relevant information are available (e.g., neighborhood demographics or nearby amenities).
4. Filter and subset the data based on specific criteria, such as a particular time period, property type, or location.
5. Handle categorical variables by encoding them appropriately (e.g., one-hot encoding or label encoding) for further analysis.
6. Aggregate the data to calculate summary statistics or derived metrics such as average sale prices by neighborhood or property type.
7. Identify and handle outliers or extreme values in the data that may affect the analysis or modeling process.

Code :

```
import pandas as pd

df=pd.read_csv("Bengaluru_House_Data.csv")
df.head()
```

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521	3.0	1.0	95.00
4	Super built-up Area	Ready To Move	Kothanur	2 BHK	NaN	1200	2.0	1.0	51.00

```
df.shape
```

(13320, 9)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13320 entries, 0 to 13319
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   area_type     13320 non-null  object
 1   availability  13320 non-null  object
 2   location      13319 non-null  object
 3   size          13304 non-null  object
 4   society       7818 non-null   object
 5   total_sqft    13320 non-null  object
 6   bath          13247 non-null  float64
 7   balcony       12711 non-null  float64
 8   price         13320 non-null  float64
dtypes: float64(3), object(6)
memory usage: 936.7+ KB
```

df.dtypes

```
area_type        object
availability     object
location         object
size             object
society          object
total_sqft       object
bath            float64
balcony         float64
price           float64
dtype: object
```

df.describe()

	bath	balcony	price
count	13247.000000	12711.000000	13320.000000
mean	2.692610	1.584376	112.565627
std	1.341458	0.817263	148.971674
min	1.000000	0.000000	8.000000
25%	2.000000	1.000000	50.000000
50%	2.000000	2.000000	72.000000
75%	3.000000	2.000000	120.000000
max	40.000000	3.000000	3600.000000

df.columns

```
Index(['area_type', 'availability', 'location', 'size', 'society',
       'total_sqft', 'bath', 'balcony', 'price'],
      dtype='object')
```

removing unnessasary features

df = df.drop(['availability','society','area_type','balcony'],axis='columns')
df.head()

	location	size	total_sqft	bath	price
0	Electronic City Phase II	2 BHK	1056	2.0	39.07
1	Chikka Tirupathi	4 Bedroom	2600	5.0	120.00
2	Uttarahalli	3 BHK	1440	2.0	62.00
3	Lingadheeranahalli	3 BHK	1521	3.0	95.00
4	Kothanur	2 BHK	1200	2.0	51.00

Checking for null values

df.isnull().sum()

```
location       1
size          16
total_sqft     0
bath          73
price          0
dtype: int64
```

as there are not more null values , we drop them directly

df.dropna(inplace=True)

df['size'].unique()

```
array(['2 BHK', '4 Bedroom', '3 BHK', '4 BHK', '6 Bedroom', '3 Bedroom',
       '1 BHK', '1 RK', '1 Bedroom', '8 Bedroom', '2 Bedroom',
       '7 Bedroom', '5 BHK', '7 BHK', '6 BHK', '5 Bedroom', '11 BHK',
       '9 BHK', '9 Bedroom', '27 BHK', '10 Bedroom', '11 Bedroom',
       '10 BHK', '19 BHK', '16 BHK', '43 Bedroom', '14 BHK', '8 BHK',
       '12 Bedroom', '13 BHK', '18 Bedroom'], dtype=object)
```

Now will separate the string by spliting it from space

df['bhk'] = df['size'].apply(lambda x : int(x.split(' ')[0]))
df.head()

	location	size	total_sqft	bath	price	bhk
0	Electronic City Phase II	2 BHK	1056	2.0	39.07	2
1	Chikka Tirupathi	4 Bedroom	2600	5.0	120.00	4
2	Uttarahalli	3 BHK	1440	2.0	62.00	3
3	Lingadheeranahalli	3 BHK	1521	3.0	95.00	3
4	Kothanur	2 BHK	1200	2.0	51.00	2

now we ahave separate column of bhk in numbers

df['bhk'].unique()

```
array([ 2,  4,  3,  6,  1,  8,  7,  5, 11,  9, 27, 10, 19, 16, 43, 14, 12,
       13, 18])
```

df[df['bhk']>=20]

	location	size	total_sqft	bath	price	bhk
1718	2Electronic City Phase II	27 BHK	8000	27.0	230.0	27
4684	Munnekollal	43 Bedroom	2400	40.0	660.0	43

total_sqft this column in data set consists of some values in 'range' form ,which is not proper way to train data so will take avg of it eg : '1133 - 1384'

df['total_sqft'].unique()

```
array(['1056', '2600', '1440', ..., '1133 - 1384', '774', '4689'],
      shape=(2067,), dtype=object)
```

Function to check whether this columns consist the record in proper format

```
def is_floadt(x):
    try:
        float(x)
    except:
        return(False)
    return(True)
```

df[~df['total_sqft'].apply(is_floadt)].head()

	location	size	total_sqft	bath	price	bhk
30	Yelahanka	4 BHK	2100 - 2850	4.0	186.000	4
122	Hebbal	4 BHK	3067 - 8156	4.0	477.000	4
137	8th Phase JP Nagar	2 BHK	1042 - 1105	2.0	54.005	2
165	Sarjapur	2 BHK	1145 - 1340	2.0	43.490	2
188	KR Puram	2 BHK	1015 - 1540	2.0	56.800	2

Now will add them and take average

```
def convert_sqft_to_num(x):
  try:
    if '-' in x:
      nums = x.split('-')
      return (float(nums[0]) + float(nums[1])) / 2
    return float(x)
  except:
    return None

df['total_sqft'] = df['total_sqft'].apply(convert_sqft_to_num)
df.head()
```

	location	size	total_sqft	bath	price	bhk
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4
2	Uttarahalli	3 BHK	1440.0	2.0	62.00	3
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3
4	Kothanur	2 BHK	1200.0	2.0	51.00	2

will now create a another column name price_per_Sqft

```
df2=df.copy()

df2['price_per_Sqft'] = df2['price']* 100000/ df2['total_sqft']
df2.head(5)
```

	location	size	total_sqft	bath	price	bhk	price_per_Sqft
0	Electronic City Phase II	2 BHK	1056.0	2.0	39.07	2	3699.810606
1	Chikka Tirupathi	4 Bedroom	2600.0	5.0	120.00	4	4615.384615
2	Uttarahalli	3 BHK	1440.0	2.0	62.00	3	4305.555556
3	Lingadheeranahalli	3 BHK	1521.0	3.0	95.00	3	6245.890861
4	Kothanur	2 BHK	1200.0	2.0	51.00	2	4250.000000

Dimensionality Curse

df2.location.unique()

```
array(['Electronic City Phase II', 'Chikka Tirupathi', 'Uttarahalli', ...,
       '12th cross srinivas nagar banshankari 3rd stage',
       'Havanur extension', 'Abshot Layout'], shape=(1304,), dtype=object)
```

so here we cant derictly aplly hot_encoding because it will generate 1304 other columns / features which is huge and not a goot practive

len(df2.location.unique())

df2.location = df2.location.apply(lambda x : x.strip())
location_stats = df2.groupby('location')['location'].agg('count').sort_values(ascending = False)
# this is giving the how much rows are assigned with per unique location
location_stats

```
location
Whitefield                          535
Sarjapur  Road                      392
Electronic City                     304
Kanakpura Road                      266
Thanisandra                         236
                                   ...
poornaprajna layout                   1
pavitra paradise                      1
near Ramanashree California resort    1
mvj engineering college               1
1Kasavanhalli                         1
Name: location, Length: 1293, dtype: int64
```

# this much values have rows less than 10 so now will make them as a 'other' section
len(location_stats[location_stats < 10])

location_stats_less_than_10 = location_stats[location_stats < 10]
location_stats_less_than_10

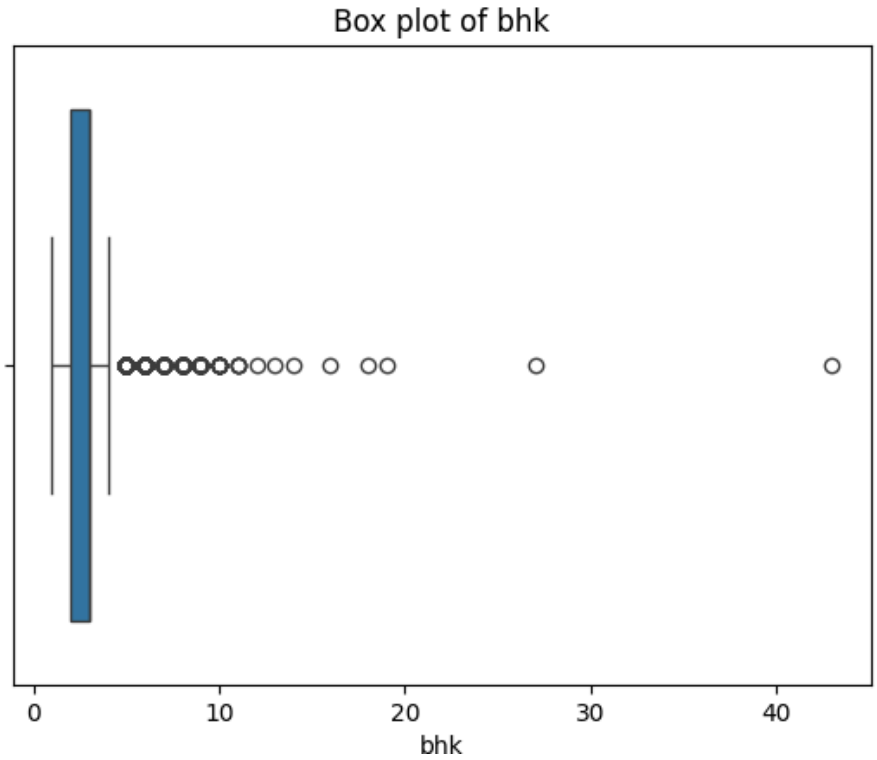
```
location
Medahalli                           9
Vishwanatha Nagenahalli             9
2nd Phase JP Nagar                  9
Chandra Layout                      9
Chennammana Kere                    9
                                   ..
poornaprajna layout                 1
pavitra paradise                    1
near Ramanashree California resort  1
mvj engineering college             1
1Kasavanhalli                       1
Name: location, Length: 1039, dtype: int64
```

df2.location = df2.location.apply(lambda x: 'other' if x in location_stats_less_than_10 else x)
len(df2.location.unique()) # less unique locations now

Outlier Removal Using Standard Deviation and Mean

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df2['bhk'])
plt.title('Box plot of bhk')
plt.show()
```




```
df3 = df2[~(df2.total_sqft / df2.bhk < 300 )]
df3.shape
```

(12502, 7)

```
df3.price_per_Sqft.describe()
```

```
count     12456.000000
mean       6308.502826
std        4168.127339
min         267.829813
25%        4210.526316
50%        5294.117647
75%        6916.666667
max      176470.588235
Name: price_per_Sqft, dtype: float64
```

Here we find that min price per sqft is 267 rs/sqft whereas max is 12000000, this shows a wide variation in property prices. We should remove outliers per location using mean and one standard deviation

```
import numpy as np
def remove_pps_outliers(df):
    df_out = pd.DataFrame()
    for key, subdf in df.groupby('location'):
        m = np.mean(subdf.price_per_Sqft)
        st = np.std(subdf.price_per_Sqft)
        reduced_df = subdf[(subdf.price_per_Sqft>(m-st)) & (subdf.price_per_Sqft<=(m+st))]
        df_out = pd.concat([df_out,reduced_df],ignore_index=True)
    return df_out


df4 = remove_pps_outliers(df3)
df4.shape
```

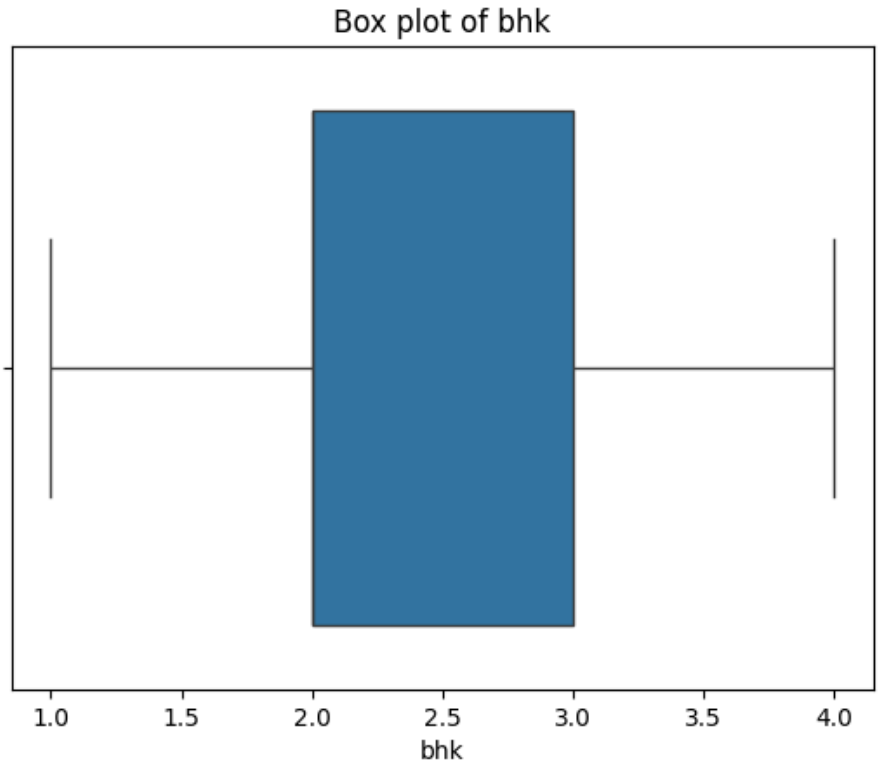
(10222, 7)

```
Q1 = df4['bhk'].quantile(0.25)
Q3 = df4['bhk'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter dataframe to keep only non-outliers
df5 = df4[(df4['bhk'] >= lower_bound) & (df4['bhk'] <= upper_bound)]


sns.boxplot(x=df5['bhk'])
plt.title('Box plot of bhk')
plt.show()
```



Encoding (One Hot)

```
dummies = pd.get_dummies(df6.location)
dummies = dummies.astype(int)
dummies.head()
```

	1st Block Jayanagar	1st Block Koramangala	1st Phase JP Nagar	2nd Phase Judicial Layout	2nd Stage Nagarbhavi	5th Block Hbr Layout	5th Phase JP Nagar	6th Phase JP Nagar	7th Phase JP Nagar	8th Phase JP Nagar	...	Vishveshwarya Layout	Vishwapriya Layout
0	1	0	0	0	0	0	0	0	0	0	...	0	0
1	1	0	0	0	0	0	0	0	0	0	...	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0
4	1	0	0	0	0	0	0	0	0	0	...	0	0

5 rows × 255 columns

```
df7 = pd.concat([df6,dummies],axis='columns')
```

Building Model

```
x = df7.drop(['price'],axis ='columns')
y = df7.price

from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2, random_state=42)


from sklearn.linear_model import LinearRegression
model = LinearRegression()

model.fit(x_train,y_train)

acc=model.score(x_test,y_test)
```

```
82.85157791954349 %
```

```
def predict_price(location,sqft,bath,bhk):
    loc_index = np.where(x.columns==location)[0][0]

    x1 = np.zeros(len(x.columns))
    x1[0] = sqft
    x1[1] = bath
    x1[2] = bhk
    if loc_index >= 0:
        x1[loc_index] = 1

    return model.predict([x1])[0]
```

```
predict_price('Kothanur',1000, 2, 2)
```

```
np.float64(41.396794469712646)
```